

Case B - Multi Crowd Exchange through same Door

Door Size	P1	P4	P5	P7
1m	8	10	26	29
2m	7	11	25	30
3m	6	12	24	31
4m	5	13	26	32

```
clear all;
clc;

%-----Domain Parameter-----%
% Defining velocity
global INFINITE eta CFL speed_A speed_B discomfort_A discomfort_B cost_A cost_B
x_velocity_A y_velocity_A x_velocity_B y_velocity_B;

% Grid Points
global Nx; Nx = 31 + 6;
global Ny; Ny = 31 + 6;

%--Important Parameter
INFINITE = 10^12;
eta = 10^-6;
error_eta = 10^-9;
CFL = 0.5;

% To track the error for each iterations
iteration_error = [];

% Dimension of Domain
Lx = 15.5;
Ly = 15.5;

% Domain
x = linspace(0,Lx,Nx);
y = linspace(0,Ly,Ny);
[X,Y] = meshgrid(x,y);

% This will be used for plotting
x_plot = linspace(0, Lx, Nx-6);
y_plot = linspace(0, Ly, Ny-6);

% Space Interval
global h;
h = Lx/(Nx-6);
```

```

% Total Time (A typical Mumbai Sub-Urban Train halts for about 15 to 30
% seconds)
t_total = 50;

% --- Obstacle modeling ----%
global door_lenght;
door_lenght = 1;

global P1 P4 P5 P7;

global obstacle;

% Make the Obstacle depending on Door Length.
makeDoorLenght(door_lenght);

%-----Initial Condition-----%

global initial_density;
initial_density = 1;

% Zero Density initial condition
rho_a = makeInitialDensity('A',initial_density); % Density of A
rho_b = makeInitialDensity('B',initial_density); % Density at B
rho_c = makeInitialDensity('C',initial_density); % Density at C

rho = rho_a + rho_b + rho_c; % Total Density at each point

%-----Multiple Crowd Description-----%
% Rho_max and V_max
rho_max_a = 10;
rho_max_b = 10;
v_max_a = 2;
v_max_b = 2;
% Discomfort
k_a = 0.002;
k_b = 0.002;

%--Function to compute Velocity using Density. Beware that this rho will be
% the total density in the domain.
function speed = makeSpeed(rho,v_max,rho_max)
    global Nx Ny;
    speed = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            speed(j,i) = v_max*(1 - rho(j,i)/rho_max);
        end
    end
end

%--Function to compute Discomfort using (rho here is the total density)

```

```

function discomfort = makeDiscomfort(rho,k)
    global Nx Ny;
    discomfort = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            discomfort(j,i) = k*rho(j,i)*rho(j,i);
        end
    end
end

%--Function to compute Cost function using Speed and Discomfort
function cost = makeCost(speed,discomfort)
    global Nx Ny;
    cost = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            cost(j,i) = 1/speed(j,i) + discomfort(j,i);
        end
    end
end

% Function that will calculate the flux depending on the u,v and Density.
% X_velocity and y_velocity will be found using the potential.
function flux = makeFlux(rho,x_velocity,y_velocity)
    global Nx Ny;

    % Initializing Flux
    flux_x = zeros(Ny,Nx);
    flux_y = zeros(Ny,Nx);
    flux = zeros(2, Ny, Nx);

    for i = 1:1:Nx
        for j = 1:1:Ny

            flux_x(j,i) = rho(j,i) * x_velocity(j,i);
            flux_y(j,i) = rho(j,i) * y_velocity(j,i);

            flux(1,j,i) = flux_x(j,i);
            flux(2,j,i) = flux_y(j,i);
        end
    end
end

%---Plots to verify the assignment

% Plotting the Density of the crowd A
figure(1);
contourf(X, Y, rho_a, 100, 'LineStyle', 'none');
colorbar;
title('Density of Crowd A');
xlabel('x');

```

```

ylabel('y');
axis equal tight;

% Plotting the density of the crowd B
figure(2);
contourf(X, Y, rho_b, 100, 'LineStyle', 'none');
colorbar;
title('Density of Crowd B');
xlabel('x');
ylabel('y');
axis equal tight;
% Plotting the total density
figure(3);
contourf(X, Y, rho, 100, 'LineStyle', 'none');
colorbar;
title('Total Density');
xlabel('x');
ylabel('y');
axis equal tight;
% Plotting the obstacle
figure(4);
imagesc(obstacle);
colorbar;
title('Obstacle Map');
axis equal tight;
% Plotting the obstacle on density
figure(5);
contourf(X, Y, rho, 100, 'LineStyle', 'none');
hold on;
contour(X, Y, obstacle, [1 1], 'k', 'LineWidth', 2);
colorbar;
title('Density with Obstacles');
xlabel('x');
ylabel('y');
axis equal tight;
hold off;
% Computing cost for the density (rho)
speed_A = makeSpeed(rho,v_max_a,rho_max_a);
speed_B = makeSpeed(rho,v_max_b,rho_max_b);
discomfort_A = makeDiscomfort(rho,k_a);
discomfort_B = makeDiscomfort(rho,k_b);
cost_A = makeCost(speed_A,discomfort_A);
cost_B = makeCost(speed_B,discomfort_B);

%-- Calculate Initial Phi guess using Lower order Fast Sweep
phi_first_order_A = First_order_Godunov_Fast_Sweep_Algorithm("A",cost_A,h,INFINITE);
phi_first_order_B = First_order_Godunov_Fast_Sweep_Algorithm("B",cost_B,h,INFINITE);
%-- Calculate The final phi using higher order scheme and above initial guess

```

```

phi_A =
weno_fast sweep("A",cost_A,h,eta,error_eta,INFINITE,phi_first_order_A,iteration_error
);
phi_B =
weno_fast sweep("B",cost_B,h,eta,error_eta,INFINITE,phi_first_order_B,iteration_error
);

figure;

% ----- First-order potentials -----
subplot(2,2,1);
phi_plot = phi_first_order_A(4:Ny-3,4:Nx-3);
phi_plot(phi_plot >= INFINITE/2) = NaN;

contourf(x_plot, y_plot, phi_plot, 40, 'LineColor','none');
set(gca,'YDir','normal');
axis equal tight;
colorbar;
title('\phi_A - First Order');

subplot(2,2,2);
phi_plot = phi_first_order_B(4:Ny-3,4:Nx-3);
phi_plot(phi_plot >= INFINITE/2) = NaN;

contourf(x_plot, y_plot, phi_plot, 40, 'LineColor','none');
set(gca,'YDir','normal');
axis equal tight;
colorbar;
title('\phi_B - First Order');

% ----- WENO potentials -----
subplot(2,2,3);
phi_plot = phi_A(4:Ny-3,4:Nx-3);
phi_plot(phi_plot >= INFINITE/2) = NaN;

contourf(x_plot, y_plot, phi_plot, 40, 'LineColor','none');
set(gca,'YDir','normal');
axis equal tight;
colorbar;
title('\phi_A - WENO');

subplot(2,2,4);
phi_plot = phi_B(4:Ny-3,4:Nx-3);
phi_plot(phi_plot >= INFINITE/2) = NaN;

contourf(x_plot, y_plot, phi_plot, 40, 'LineColor','none');
set(gca,'YDir','normal');
axis equal tight;
colorbar;

```

```

title('\phi_B - WENO');

sgtitle(sprintf('Initial Potentials | Door Length %.2f | Density %.2f', ...
                door_lenght, initial_density));

drawnow;

for set_door = 2:1:4

    % Update the door size
    door_lenght = set_door;
    % Update the Obstacle Boolean
    makeDoorLenght(door_lenght);

    for set_density = 0.5:0.5:9.5

        %--Pre allocating the memory (Refresh the Density)
        rho_n_A = zeros(Ny,Nx);    rho_n_B = zeros(Ny,Nx);
        rho_1_A = zeros(Ny,Nx);    rho_1_B = zeros(Ny,Nx);
        rho_2_A = zeros(Ny,Nx);    rho_2_B = zeros(Ny,Nx);
        rho_next_A = zeros(Ny,Nx); rho_next_B = zeros(Ny,Nx);

        % Update the Initial Density
        initial_density = set_density;
        % Update the initial density
        rho_a = makeInitialDensity('A',initial_density);
        rho_b = makeInitialDensity('B',initial_density);
        rho_c = makeInitialDensity('C',initial_density);

        % Stopping condition
        stop_low = -1;
        stop_high = 11;

        %-----MAIN SOLVER-----%
        t = 0;
        step = 0; % For plotting at regular interval

        % Video Capturing
        file_name = sprintf('Density_Contour_D_%.2f_rho_%.2f.mp4', door_lenght,
initial_density);
        v = VideoWriter(file_name, 'MPEG-4');
        open(v);

        % Time Marching
        while t < t_total

            % Coupling Term
            rho = rho_a + rho_b + rho_c;

```

```

% Computing cost for the density (rho)
speed_A = makeSpeed(rho,v_max_a,rho_max_a);
speed_B = makeSpeed(rho,v_max_b,rho_max_b);
discomfort_A = makeDiscomfort(rho,k_a);
discomfort_B = makeDiscomfort(rho,k_b);
cost_A = makeCost(speed_A,discomfort_A);
cost_B = makeCost(speed_B,discomfort_B);

%-- Calculate Initial Phi guess using Lower order Fast Sweep
phi_first_order_A =
First_order_Godunov_Fast_Sweep_Algorithm("A",cost_A,h,INFINITE);
phi_first_order_B =
First_order_Godunov_Fast_Sweep_Algorithm("B",cost_B,h,INFINITE);
%-- Calculate The final phi using higher order scheme and above initial
guess
phi_A =
weno_fastsweep("A",cost_A,h,eta,error_eta,INFINITE,phi_first_order_A,iteration_error
);
phi_B =
weno_fastsweep("B",cost_B,h,eta,error_eta,INFINITE,phi_first_order_B,iteration_error
);

%--Calculate the Components of Velocity
makeComponent("A",phi_A,speed_A);
makeComponent("B",phi_B,speed_B);

% Updating the delta_t with proper CFL condition (Stochastic paper)
u_max = max([abs(x_velocity_A(:)); abs(x_velocity_B(:))]);
v_max = max([abs(y_velocity_A(:)); abs(y_velocity_B(:))]);
% dt = CFL / ( ( u_max/h ).^(5/3) + ( v_max/h ).^(5/3) + 1e-12 );
dt = CFL * h / ( ( u_max ) + ( v_max ) + 1e-12 );

% ----- Making rho_n and RHS of rho_n (Crowd A and B)-----%
rho_n_A = rho_a;
rho_n_B = rho_b;

semi_discritze_form_rho_n_A = makeRHS("A",rho_n_A,t);
semi_discritze_form_rho_n_B = makeRHS("B",rho_n_B,t);

% Check for the negative rho_n values
if (any(rho_n_A(:) < stop_low)|| any(rho_n_B(:) < stop_low))
    break;
elseif (any(rho_n_A(:) > stop_high) || any(rho_n_B(:) > stop_high))
    break;
end

% Extrapolation
rho_n_A = extrapolate_Density("A",rho_n_A);
rho_n_B = extrapolate_Density("B",rho_n_B);

```

```

% ----- Making the RHO_1 and RHS of rho_1 (Crowd A and B)----- %
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_1_A(j,i) = rho_n_A(j,i) + dt *
semi_discritze_form_rho_n_A(j-3,i-3);
            rho_1_B(j,i) = rho_n_B(j,i) + dt *
semi_discritze_form_rho_n_B(j-3,i-3);
        else
            rho_1_A(j,i) = 0;
            rho_1_B(j,i) = 0;
        end
    end
end

% -- Extrapolation (My Intuition)
rho_1_A = extrapolate_Density("A",rho_1_A);
rho_1_B = extrapolate_Density("B",rho_1_B);

% -- RHS of discretized form using rho_1
semi_discritze_form_rho_1_A = makeRHS("A",rho_1_A,t);
semi_discritze_form_rho_1_B = makeRHS("B",rho_1_B,t);

% Check for the negative rho_1 values
if (any(rho_1_A(:) < stop_low) || any(rho_1_B(:) < stop_low))
    break
elseif (any(rho_n_A(:) > stop_high) || any(rho_n_B(:) > stop_high))
    break;
end

%----- Making the RHO_2 and RHS of RHO_2 -----%
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_2_A(j,i) = 3/4 * rho_n_A(j,i) + 1/4 * (rho_1_A(j,i) +
dt * semi_discritze_form_rho_1_A(j-3,i-3));
            rho_2_B(j,i) = 3/4 * rho_n_B(j,i) + 1/4 * (rho_1_B(j,i) +
dt * semi_discritze_form_rho_1_B(j-3,i-3));
        else
            rho_2_A(j,i) = 0;
            rho_2_B(j,i) = 0;
        end
    end
end
end

```

```

% -- Extrapolation (My Intuition)
rho_2_A = extrapolate_Density("A",rho_2_A);
rho_2_B = extrapolate_Density("B",rho_2_B);

% -- RHS of discretized form using rho_2
semi_discritze_form_rho_2_A = makeRHS("A",rho_2_A,t);
semi_discritze_form_rho_2_B = makeRHS("B",rho_2_B,t);

% Check for the negative rho_2 values
if (any(rho_2_A(:) < stop_low) || any(rho_2_B(:) < stop_low))
    break;
elseif (any(rho_2_A(:) > stop_high) || any(rho_2_B(:) > stop_high))
    break;
end

%----- Making the rho_next (Crowd A and B)-----%

for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_next_A(j,i) = 1/3 * rho_n_A(j,i) + 2/3 * (rho_2_A(j,i)
+ dt * semi_discritze_form_rho_2_A(j-3,i-3));
            rho_next_B(j,i) = 1/3 * rho_n_B(j,i) + 2/3 * (rho_2_B(j,i)
+ dt * semi_discritze_form_rho_2_B(j-3,i-3));
        else
            rho_next_A(j,i) = 0;
            rho_next_B(j,i) = 0;
        end
    end
end

% -- Extrapolation (My Intuition)
rho_next_A = extrapolate_Density("A",rho_next_A);
rho_next_B = extrapolate_Density("B",rho_next_B);

% Substituting back the rho value
rho_a = rho_next_A;
rho_b = rho_next_B;
rho = rho_next_A + rho_next_B;

%--Check if any of the density values is equal to zero
if (any(rho_next_A(:) < stop_low) || any(rho_next_B(:) < stop_low))
    break;
elseif (any(rho_next_A(:) > stop_high) || any(rho_next_B(:) >
stop_high))
    break;
end

```

```

if mod(step,2) == 0

    figure(6);
    clf;

    % --- Custom Color Maps (White to Pure Color) ---
    % Crowd A : White -> Bright Blue
    blueMap = [linspace(1,0,256)', linspace(1,0.4,256)',
linspace(1,1,256)'];

    % Crowd B : White -> Bright Red
    redMap = [linspace(1,1,256)', linspace(1,0,256)',
linspace(1,0,256)'];

    % Extract density matrices to avoid repetitive indexing
    density_A = rho_a(4:Ny-3, 4:Nx-3);
    density_B = rho_b(4:Ny-3, 4:Nx-3);

    %% =====
    % Layer 1: Crowd A (Blue)
    % =====
    ax1 = axes;

    h1 = imagesc(x_plot, y_plot, density_A);
    set(gca, 'YDir', 'normal');
    set(h1, 'Interpolation', 'bilinear');

    % Apply Transparency to Crowd A so it blends with B
    alphaData_A = min(density_A / 5, 1); % Fully opaque at density = 5
    set(h1, 'AlphaData', alphaData_A);

    axis equal tight;
    xlim([0 Lx]); ylim([0 Ly]);
    clim([0 10]);
    colormap(ax1, blueMap);

    cb1 = colorbar(ax1);
    cb1.Label.String = '\rho_A';

    xlabel('x');
    ylabel('y');

    %% =====
    % Layer 2: Crowd B (Red)
    % =====
    ax2 = axes;

    h2 = imagesc(x_plot, y_plot, density_B);
    set(gca, 'YDir', 'normal');
    set(h2, 'Interpolation', 'bilinear');

```

```

% Apply Transparency to Crowd B
alphaData_B = min(density_B / 5, 1);
set(h2, 'AlphaData', alphaData_B);

% Make the second axis completely transparent and hide its labels
set(ax2, 'Color', 'none', 'XTick', [], 'YTick', []);

axis equal tight;
xlim([0 Lx]); ylim([0 Ly]);
clim([0 10]);
colormap(ax2, redMap);

% Match position to perfectly align the visual layers
ax2.Position = ax1.Position;

% Second colorbar setup
cb2 = colorbar(ax2);
cb2.Label.String = '\rho_B';

% Offset the second colorbar to the right so it doesn't overlap cb1
% (Adding space prevents it from shifting over the main plot area)
cb2.Position(1) = ax1.Position(1) + ax1.Position(3) + 0.08;

%% =====
% Title & Rendering
% =====
sgtitle(sprintf('Door Length : %.2f; Initial Density : %.2f; t =
%.2f;', door_lenght , initial_density, t));

drawnow;

%% =====
% Save frame
% =====
frame = getframe(gcf);
writeVideo(v, frame);

end

% Update the time step
t = t + dt;
step = step + 1;

end
% Close the Video
close(v);

end
end

```

```

% function to set the door length
function makeDoorLenght(length)
    global obstacle Nx Ny;
    global P1 P4 P5 P7;
    switch length
        case 1
            P1 = 8; P4 = 10; P5 = 28; P7 = 30;
        case 2
            P1 = 7; P4 = 11; P5 = 27; P7 = 31;
        case 3
            P1 = 6; P4 = 12; P5 = 26; P7 = 32;
        case 4
            P1 = 5; P4 = 13; P5 = 25; P7 = 33;
    end

    obstacle = false(Ny,Nx);

    obstacle(1:P1,18:19) = true; % Obstacle 1
    obstacle(P4:P5,18:19) = true; % Obstacle 2
    obstacle(P7:Ny,18:19) = true; % Obstacle 3
end

```

Initial Condition on the density

So there are two kinds of crowd we are dealing with. The first one which is to left and, wants to move to right side of the domain, and the other one which is at the left and wants to move to the left side of the domain. Also I want to make the

```

function rho = makeInitialDensity(type,initial_density)
    global Ny Nx;
    switch type
        % If the crowd is of the platform waiting for the train.
        case 'A'
            % Crowd at Left Side (Red)
            rho = zeros(Ny,Nx); % First make this zero
            rho(4:Ny-3,11:17) = initial_density;
        case 'B'
            % Crowd at Right side (Blue)
            rho = zeros(Ny,Nx);
            rho(4:Ny-3,20:26) = initial_density;
        case 'C'
            % No need of Additional Crowd
            rho = zeros(Ny,Nx);
    end
end

```

Components of Velocity

```

function makeComponent(type,phi,speed)

```

```

global Nx Ny eta h x_velocity_A y_velocity_A x_velocity_B y_velocity_B;

[phi_x, phi_y] = gradient(phi, h, h); % partial derivatives of phi in x and y
mag_phi = sqrt(phi_x.^2 + phi_y.^2);
mag_phi = max(mag_phi, eta);          % To be safe with denominator

switch type
    case "A"
        for i = 1:1:Nx
            for j = 1:1:Ny
                x_velocity_A(j,i) = speed(j,i) * (- phi_x(j,i))/mag_phi(j,i);
                y_velocity_A(j,i) = speed(j,i) * (- phi_y(j,i))/mag_phi(j,i);
            end
        end
    case "B"
        for i = 1:1:Nx
            for j = 1:1:Ny
                x_velocity_B(j,i) = speed(j,i) * (- phi_x(j,i))/mag_phi(j,i);
                y_velocity_B(j,i) = speed(j,i) * (- phi_y(j,i))/mag_phi(j,i);
            end
        end
end
end
end

```

First order Godunov Fast Sweep Algorithm (Older Version)

```

function phi = First_order_Godunov_Fast_Sweep_Algorithm(type,c,h,INFINITE)
global Nx Ny obstacle;
phi = INFINITE*ones(Ny, Nx);    % Initial potential field

% Exit will depend on the type of crowd
switch type
    case "A"
        phi(4:Ny-3,Nx-3) = 0; % Right Boundary
    case "B"
        phi(4:Ny-3,4) = 0; % Left Boundary
end

% Sweep Iterations
for sweep = 1:20

    % Sweep 1:
    for i = 2:Nx-1
        for j = 2:Ny-1
            if ~obstacle(j, i)
                phi = update_phi(phi, c, i, j, h);
            else
                phi(j,i) = INFINITE;
            end
        end
    end
end

```

```

end

% Extrapolation of the phi values
phi = extrapolate_phi(type,phi);

% Sweep 2:
for i = Nx-1:-1:2
    for j = 2:Ny-1
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Extrapolation of the phi values
phi = extrapolate_phi(type,phi);

% Sweep 3:
for i = Nx-1:-1:2
    for j = Ny-1:-1:2
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Extrapolation of the phi values
phi = extrapolate_phi(type,phi);

% Sweep 4:
for i = 2:Nx-1
    for j = Ny-1:-1:2
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Extrapolation of the phi values
phi = extrapolate_phi(type,phi);

end
end

```

```
% Subfunction: Update one grid point using Godunov scheme
```

```
function phi = update_phi(phi, c, i, j, dx)
    a = min(phi(j, i-1), phi(j, i+1)); % x-direction neighbors
    b = min(phi(j-1, i), phi(j+1, i)); % y-direction neighbors
    c_i = c(j, i); % Local inverse speed
    if abs(a - b) >= c_i * dx
        phi(j, i) = min(a, b) + c_i * dx;
    else
        inside = 2 * (c_i * dx)^2 - (a - b)^2;
        if inside >= 0
            phi(j, i) = (a + b + sqrt(inside)) / 2;
        end
    end
end
```

WENO FAST SWEEP

```
function phi =
weno_fast_sweep(type,c,h,eta,error_eta,INFINITE,initial_guess,iteration_error)

    global Nx Ny obstacle;

    % Phi is initialized with infinity
    phi = initial_guess;
    phi_old = INFINITE * ones(size(phi));

    % Iterations
    iterations = 0;
    loop_safety = 0;

    % Iterations
    while (sum(abs(phi(:)-phi_old(:))) > error_eta && loop_safety<1000)

        %fprintf("Iteration = %d ",iterations);
        err = sum(abs(phi(:)-phi_old(:)));
        %fprintf("Error = %0.10f\n",err);

        phi_old = phi;
        iterations = iterations + 1;
        iteration_error(iterations) = err;
        loop_safety = loop_safety + 1;

        for sweep = 1:4
            switch sweep
                case 1
                    ix = 4:1:Nx-3; jy = 4:1:Ny-3;
                case 2
                    ix = 4:1:Nx-3; jy = Ny-3:-1:4;
            end
        end
    end
```

```

case 3
    ix = Nx-3:-1:4; jy = 4:1:Ny-3;
case 4
    ix = Nx-3:-1:4; jy = Ny-3:-1:4;
end

for i = ix
    for j = jy
        %if the point is outside the obstacle
        if(~obstacle(j,i))
            %-----Calculating the phix_min-----_%

            r_back = (eta + (phi(j,i) - 2*phi(j,i-1) + phi(j,i-2))^2)/
(eta + (phi(j,i+1) - 2*phi(j,i) + phi(j,i-1))^2);
            r_front = (eta + (phi(j,i) - 2*phi(j,i+1) + phi(j,i+2))^2)/
(eta + (phi(j,i+1) - 2*phi(j,i) + phi(j,i-1))^2);

            % Calculate the w
            w_front = 1/(1+2*(r_front^2));
            w_back = 1/(1+2*(r_back^2));

            % Dell phi by Dell x plus and minus
            phix_minus = (1-w_back)*((phi(j,i+1)-phi(j,i-1))/(2*h)) +
w_back*((3*phi(j,i) - 4*phi(j,i-1) + phi(j,i-2))/(2*h));
            phix_plus = (1-w_front)*((phi(j,i+1)-phi(j,i-1))/(2*h)) +
w_front*((-3*phi(j,i)+4*phi(j,i+1)-phi(j,i+2))/(2*h));

            phix_min = min((phi(j,i) - h*phix_minus),(phi(j,i) +
h*phix_plus));

            %-----calculating the phiy_min-----_%

            r_back = (eta + (phi(j,i) - 2*phi(j-1,i) + phi(j-2,i))^2) /
(eta + (phi(j+1,i) - 2*phi(j,i) + phi(j-1,i))^2);
            r_front = (eta + (phi(j,i) - 2*phi(j+1,i) +
phi(j+2,i))^2) / (eta + (phi(j+1,i) - 2*phi(j,i) + phi(j-1,i))^2);

            % Calculate the w
            w_front = 1 / (1 + 2*(r_front^2));
            w_back = 1 / (1 + 2*(r_back^2));

            % Dell phi by Dell x plus and minus
            phiy_minus = (1-w_back) * ((phi(j+1,i)-phi(j-1,i))/(2*h)) +
w_back * ((3*phi(j,i) - 4*phi(j-1,i) + phi(j-2,i))/(2*h));
            phiy_plus = (1-w_front) * ((phi(j+1,i) - phi(j-1,i))/(2*h))
+ w_front * ((-3*phi(j,i) + 4*phi(j+1,i) - phi(j+2,i))/(2*h));

```

```

        phiy_min = min((phi(j,i) - h*phiy_minus), (phi(j,i) +
h*phiy_plus));

        if abs(phix_min - phiy_min) >= c(j,i)*h
            phi(j,i) = min(phiy_min,phix_min) + c(j,i)*h;
        else
            phi(j,i) = ((phix_min + phiy_min) + (2*(c(j,i)*h)^2 -
(phix_min - phiy_min)^2)^0.5)/2;
        end

        % Enforce the value of phi to be zero at exits depending on
the crowd

        switch type
        case "A"
            % Right Boundary
            if(i == Nx-3 && (j >= 4 && j <= Ny-3))
                phi(j,i) = 0;
            end
        case "B"
            % Left Boundary
            if(i == 4 && (j >= 4 && j <= Ny-3))
                phi(j,i) = 0;
            end
        end

        % If the point is inside the obstacle
    else
        phi(j,i) = INFINITE;
    end
end
end
end

    % Before each sweep I would extrapolate the phi
    phi = extrapolate_phi(type,phi);
end
end

```

Extrapolate Phi

```

function phi = extrapolate_phi(type,phi)
    global Ny Nx;
    % Bottom Edge    (3 Ghost Points)
    phi(3,:) = phi(4,:);
    phi(2,:) = phi(3,:);
    phi(1,:) = phi(2,:);

    % Top Edge      (3 Ghost Points)
    phi(Ny-2,:) = phi(Ny-3,:);

```

```

phi(Ny-1,:) = phi(Ny-2,:);
phi(Ny,:) = phi(Ny-1,:);

% Right Edge      (3 Ghost Points)
% switch type
%     case 'A'
%         phi(:,Nx-2) = -phi(:,Nx-4);
%         phi(:,Nx-1) = -phi(:,Nx-5);
%         phi(:,Nx) = -phi(:,Nx-6);
%     case 'B'
%         phi(:,Nx-2) = phi(:,Nx-3);
%         phi(:,Nx-1) = phi(:,Nx-2);
%         phi(:,Nx) = phi(:,Nx-1);
% end

phi(:,Nx-2) = phi(:,Nx-3);
phi(:,Nx-1) = phi(:,Nx-2);
phi(:,Nx) = phi(:,Nx-1);

% Left Edge      (3 Ghost Points)
% switch type
%     case 'A'
%         phi(:,3) = phi(:,4);
%         phi(:,2) = phi(:,3);
%         phi(:,1) = phi(:,2);
%     case 'B'
%         phi(:,3) = -phi(:,5);
%         phi(:,2) = -phi(:,6);
%         phi(:,1) = -phi(:,7);
% end

phi(:,3) = phi(:,4);
phi(:,2) = phi(:,3);
phi(:,1) = phi(:,2);
end

```

RHS of Semi Discretize Form

```

function semi_discrete_form = makeRHS(type,rho,t)
    global Nx Ny x_velocity_A y_velocity_A x_velocity_B y_velocity_B h obstacle;
    global P1 P4 P5 P7;

    semi_discrete_form = zeros(Ny-6,Nx-6);

    % Flux for the given rho distribution (Will depend on the type of crowd)
    switch type
        case "A"
            x_velocity = x_velocity_A;
            y_velocity = y_velocity_A;
        case "B"

```

```

        x_velocity = x_velocity_B;
        y_velocity = y_velocity_B;
    end
    flux = makeFlux(rho, x_velocity, y_velocity);
    flux_x = squeeze(flux(1,:,:));
    flux_y = squeeze(flux(2,:,:));

% Calculating the maximum speed (multiplying by 1.5 as suggested)
alpha_x = max(abs(x_velocity(:))) * 1.5;
alpha_y = max(abs(y_velocity(:))) * 1.5;

% -- Calculating the value of f+ and f- at {i+2,i+1,i,i-1,i-2,i-3} -- %
fx_negative_whole = zeros(Ny,Nx);
fx_positive_whole = zeros(Ny,Nx);
fy_negative_whole = zeros(Ny,Nx);
fy_positive_whole = zeros(Ny,Nx);

for j = 1:1:Ny
    for i=1:1:Nx
        fx_positive_whole(j,i) = 1/2 * (flux_x(j,i) + alpha_x*rho(j,i));
        fx_negative_whole(j,i) = 1/2 * (flux_x(j,i) - alpha_x*rho(j,i));
        fy_positive_whole(j,i) = 1/2 * (flux_y(j,i) + alpha_y*rho(j,i));
        fy_negative_whole(j,i) = 1/2 * (flux_y(j,i) - alpha_y*rho(j,i));
    end
end

% -- Computing the RHS of Semi Discrete form.
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        % compute only at the points where obstacle doesn't exist
        if(~obstacle(j,i))
            %--X--%

            % fx{+}_{[i+1/2,j]} - Given in the paper
            fx_plus_x_half_forward = stencil( ...
                fx_positive_whole(j,i-2), ...
                fx_positive_whole(j,i-1), ...
                fx_positive_whole(j,i), ...
                fx_positive_whole(j,i+1), ...
                fx_positive_whole(j,i+2));

            % fx{+}_{[i-1/2,j]} - By substituting i -> (i-1) in fx{+}_{[i+1/2,j]}
            fx_plus_x_half_backward = stencil( ...
                fx_positive_whole(j,i-3), ...
                fx_positive_whole(j,i-2), ...
                fx_positive_whole(j,i-1), ...
                fx_positive_whole(j,i), ...
                fx_positive_whole(j,i+1));
        end
    end
end

```

```

% fx{-}_[i+1/2,j] ( mirror of fx{+}_[i+1/2,j] )
fx_minus_x_half_forward = stencil( ...
    fx_negative_whole(j,i+3), ...
    fx_negative_whole(j,i+2), ...
    fx_negative_whole(j,i+1), ...
    fx_negative_whole(j,i), ...
    fx_negative_whole(j,i-1));

% fx{-}_[i-1/2,j] ( mirror of fx{+}_[i-1/2,j] )
fx_minus_x_half_backward = stencil( ...
    fx_negative_whole(j,i+2), ...
    fx_negative_whole(j,i+1), ...
    fx_negative_whole(j,i), ...
    fx_negative_whole(j,i-1), ...
    fx_negative_whole(j,i-2));

%--Y--%

% fy{+}_[i,j+1/2]
fy_plus_y_half_forward = stencil( ...
    fy_positive_whole(j-2,i), ...
    fy_positive_whole(j-1,i), ...
    fy_positive_whole(j,i), ...
    fy_positive_whole(j+1,i), ...
    fy_positive_whole(j+2,i));

% fy{+}_[i,j-1/2]
fy_plus_y_half_backward = stencil( ...
    fy_positive_whole(j-3,i), ...
    fy_positive_whole(j-2,i), ...
    fy_positive_whole(j-1,i), ...
    fy_positive_whole(j,i), ...
    fy_positive_whole(j+1,i));

% fy{-}_[i,j+1/2]
fy_minus_y_half_forward = stencil( ...
    fy_negative_whole(j+3,i), ...
    fy_negative_whole(j+2,i), ...
    fy_negative_whole(j+1,i), ...
    fy_negative_whole(j,i), ...
    fy_negative_whole(j-1,i));

% fy{-}_[i,j-1/2]
fy_minus_y_half_backward = stencil( ...
    fy_negative_whole(j+2,i), ...
    fy_negative_whole(j+1,i), ...
    fy_negative_whole(j,i), ...
    fy_negative_whole(j-1,i), ...
    fy_negative_whole(j-2,i));

```

```

        fx_positive_half = fx_plus_x_half_forward + fx_minus_x_half_forward;
        fx_negative_half = fx_plus_x_half_backward +
fx_minus_x_half_backward;
        fy_positive_half = fy_plus_y_half_forward + fy_minus_y_half_forward;
        fy_negative_half = fy_plus_y_half_backward +
fy_minus_y_half_backward;

    %--Boundary Condition -- %

    if(j == Ny-3) % Top Edge
        fy_positive_half = 0;
    elseif(j == 4) % Bottom Edge
        fy_negative_half = 0;
    end

    if(i == 4) % Left Edge
        switch type
            % Crowd A has a wall
            case 'A'
                fx_negative_half = 0;
            % Crowd B has the outflow
            case 'B'
                fx_negative_half = flux_x(j,4);
            end
        elseif(i == Nx-3) % Right Edge
            switch type
                % Crowd A has outflow
                case 'A'
                    fx_positive_half = flux_x(j,Nx-3);
                % Crowd B has wall
                case 'B'
                    fx_positive_half = 0;
            end
        end

    % BOUNDARY OF OBSTACLE

    % Obstacle 1
    if(j >= 1 && j <= P1)
        if(i == 17)
            fx_positive_half = 0;
        elseif(i == 20)
            fx_negative_half = 0;
        end
    elseif (j == (P1+1) && (i >= 18 && i <= 19))

        fy_negative_half = 0;
    end

```

```

% Obstacle 2
if(j >= P4 && j <= P5)
    if(i == 17)
        fx_positive_half = 0;
    elseif(i == 20)
        fx_negative_half = 0;
    end
elseif (j == (P4-1) && (i >= 18 && i <= 19))
    fy_positive_half = 0;
elseif (j == (P5+1) && (i >= 18 && i <= 19))
    fy_negative_half = 0;
end

% Obstacle 3
if(j >= P7 && j <= Ny)
    if(i == 17)
        fx_positive_half = 0;
    elseif(i == 20)
        fx_negative_half = 0;
    end
elseif (j == (P7-1) && (i >= 18 && i <= 19))
    fy_positive_half = 0;
end

% Computing RHS of Semi Discretized form
semi_discrete_form(j-3,i-3) = - 1/h * (fx_positive_half -
fx_negative_half) - 1/h * (fy_positive_half - fy_negative_half);
else
end
end
end
end
end

```

WENO Advection Stencil

```

% To compute f half. Order of input matters
function f = stencil(f1,f2,f3,f4,f5)

eta = 10^-6;

% Compute S values
s_1 = ((1/3) * f1) + ((-7/6) * f2) + ((11/6) * f3);
s_2 = ((-1/6) * f2) + ((5/6) * f3) + ((1/3) * f4);
s_3 = ((1/3) * f3) + ((5/6) * f4) + ((-1/6) * f5);

% Compute Beta
beta_1 = 13/12 * (f1 - 2*f2 + f3)^2 + 1/4 * (f1 - 4*f2 + 3*f3)^2;
beta_2 = 13/12 * (f2 - 2*f3 + f4)^2 + 1/4 * (f2 - f4)^2;

```

```

beta_3 = 13/12 * (f3 - 2*f4 + f5)^2 + 1/4 * (3*f3 - 4*f4 + f5)^2;

% Defining Gamma
gamma_1 = 1/10;
gamma_2 = 3/5;
gamma_3 = 3/10;

% Computing weights bar
w_1_dash = ( gamma_1 ) ./ ((eta + beta_1)^2);
w_2_dash = ( gamma_2 ) ./ ((eta + beta_2)^2);
w_3_dash = ( gamma_3 ) ./ ((eta + beta_3)^2);

% Computing Weights
w_1 = ( w_1_dash ) / (w_1_dash + w_2_dash + w_3_dash);
w_2 = ( w_2_dash ) / (w_1_dash + w_2_dash + w_3_dash);
w_3 = ( w_3_dash ) / (w_1_dash + w_2_dash + w_3_dash);

% Computing Flux
f = w_1 * s_1 + w_2 * s_2 + w_3 * s_3;
end

```

Density Extrapolation

```

function rho = extrapolate_Density(type,rho)
    global Ny Nx;
    % Left Edge      (3 Ghost Points)
    rho(:,3) = rho(:,4);
    rho(:,2) = rho(:,3);
    rho(:,1) = rho(:,2);
    % Bottom Edge    (3 Ghost Points)
    rho(3,:) = rho(4,:);
    rho(2,:) = rho(3,:);
    rho(1,:) = rho(2,:);
    % Top Edge       (3 Ghost Points)
    rho(Ny-2,:) = rho(Ny-3,:);
    rho(Ny-1,:) = rho(Ny-2,:);
    rho(Ny,:) = rho(Ny-1,:);
    % Right Edge
    rho(:,Nx-2) = rho(:,Nx-3);
    rho(:,Nx-1) = rho(:,Nx-2);
    rho(:,Nx) = rho(:,Nx-1);
end

```